

openagent-frontend — Datasheet

Reference document for building on top of, or integrating with, openagent-frontend.
Intended audience: **openagent-api**, **openagent-logger**, and any other service in the OpenAgent system that needs to understand what openagent-frontend is, what it owns, and how it talks to the rest of the system.

Quick Reference

Item	Value
Role	User-facing chat UI for OpenAgent
Framework	Streamlit
Language	Python 3.11
Protocol out	HTTP/1.1 + Server-Sent Events (SSE consumer)
Protocol in (user)	HTTPS (Streamlit web UI)
Host port	8000
Container port	8501 (Streamlit default)
Backend	openagent-api (<code>OPENAGENT_API_URL</code> , typically <code>:8001</code>)
Auth out	<code>X-API-Key: OPENAGENT_API_KEY</code> on every <code>/chat</code> and <code>/health</code> call
Auth in (user)	None (single-tenant)
Session store	In-memory <code>st.session_state</code> , per browser tab
Persistent store	None (stateless across browser refresh)
System prompt	Owned by openagent-api — not present in this repo
SSE decoder	<code>src/frontend/sse_decoder.py</code>
Version	1.0.0

Overview

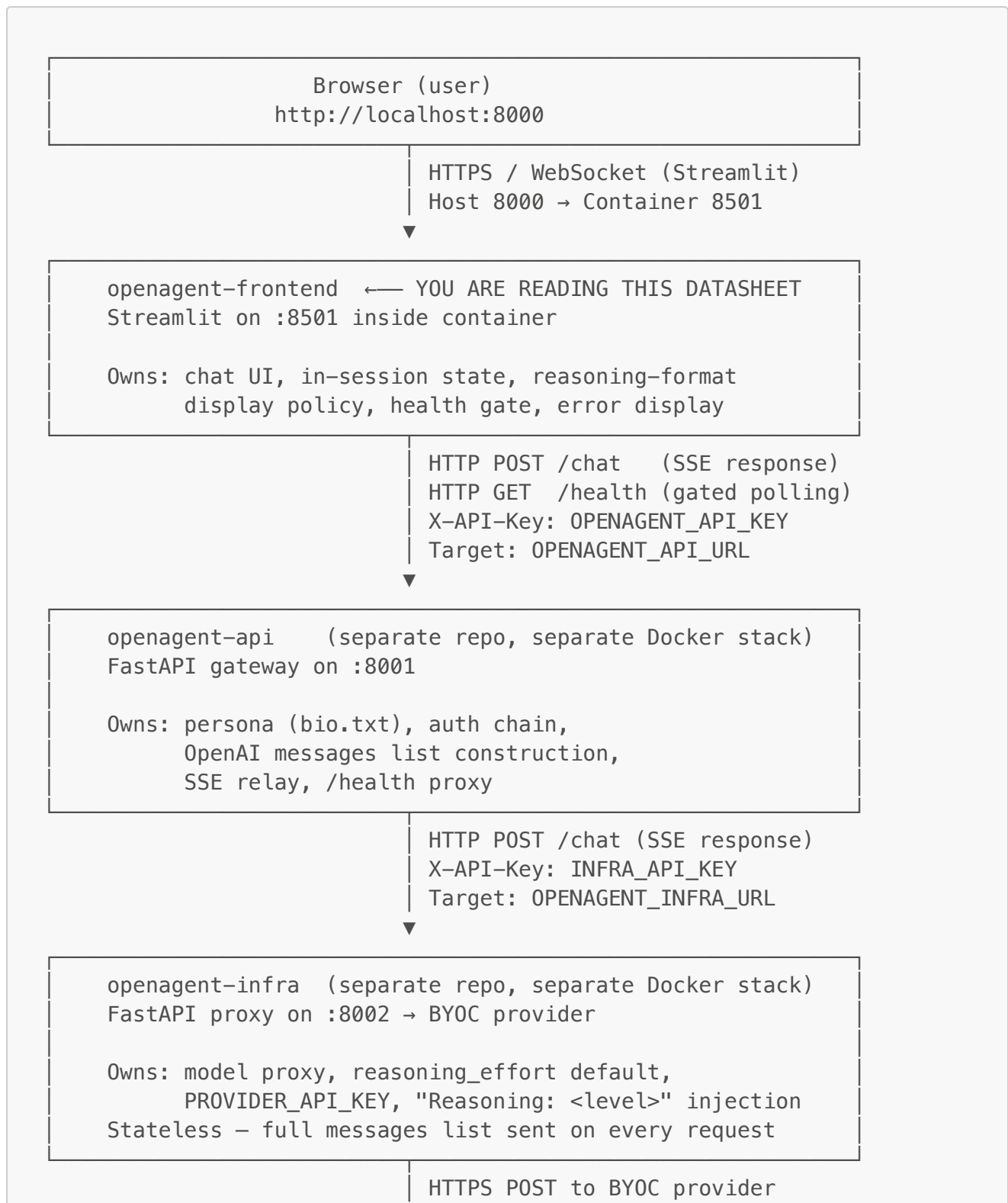
openagent-frontend is the **user interface layer** of the OpenAgent system. It is a lean Streamlit web app that renders the chat experience, tracks in-session conversation state, and consumes a streaming response from **openagent-api**. That is the entire job.

It does not own the persona. It does not authenticate to the model layer. It does not load a model, run inference, host a database, log events, or assemble the OpenAI messages list with a system prompt. **openagent-api** owns those concerns — and **openagent-frontend** is correspondingly small.

Everything that makes OpenAgent *look and feel* a certain way from the user's perspective — the chat bubbles, the streaming reasoning expander, the error banners, the health gate — happens in this service. Everything that makes the agent *be* what it is on the wire (the persona, the auth chain, the messages-list construction, the SSE relay) happens upstream in **openagent-api**.

That split is the point: the frontend is a UI, **openagent-api** is the gateway, **openagent-infra** is the model proxy, and a BYOC provider is the inference layer.

Where This Service Fits



| Authorization: Bearer PROVIDER_API_KEY



BYOC Compute Provider (e.g. RunPod, OpenAI, local vLLM)
base reasoning model
Scales to zero when idle (serverless deployments)

Port topology:

User → openagent-frontend (:8000) → openagent-api (:8001) → openagent-infra (:8002) → BYOC provider

`openagent-frontend` is the **only** client of `openagent-api`. `openagent-api` is the **only** thing `openagent-frontend` talks to over HTTP. The frontend has no knowledge of `openagent-infra` or the compute provider and never sees their auth credentials.

What This Service Owns

A strict list of responsibilities that live inside `openagent-frontend` and nowhere else. Other services should defer to this service for these concerns and must not reimplement them.

1. Chat UI rendering

The visual surface — chat bubbles, the collapsible "Show thinking" expander, the streaming token rendering with the cursor glyph, the input field, the page header, the divider, the reasoning-effort toggle — all live here. Streamlit primitives (`st.chat_message`, `st.expander`, `st.markdown`, `st.empty`, `st.chat_input`, `st.radio`) drive the layout. No other service renders UI.

2. In-session conversation state

Conversation history for the current browser tab lives in `st.session_state.messages`, a list of `{"role", "content"}` dicts. There is no `"think"` field — the reasoning chain is rendered live during streaming and not persisted back into history. State is:

- **Ephemeral** — destroyed when the tab closes or Streamlit reruns a fresh session
- **Client-side only** — never sent to or stored in any backend service
- **The source of truth for the UI** during a single session
- **Sent in full** to `openagent-api` on every `/chat` call as user/assistant turns only — `openagent-api` prepends the system message server-side

Closing the tab loses everything; there is no persistence layer.

3. Reasoning-format display policy

How the reasoning chain is presented to the user is a UX decision that lives at this layer. `openagent-frontend` uses a collapsible `st.expander("🧠 Show thinking")` rendered above the main chat bubble. Reasoning streams into the expander in real time as `delta.reasoning` tokens arrive; the visible answer streams into the main bubble as `delta.content` tokens arrive.

Note the boundary: the frontend chooses the *display policy* (where to render reasoning vs content, whether to collapse the expander by default, whether to show it at all on a given deployment). It does NOT do the *parsing* — that is owned by `sse_decoder.py`, which yields typed events with `kind="reasoning"` or `kind="content"` and lets this layer route them. For a public-facing deployment, hiding the reasoning expander entirely is a one-line change in this file; no upstream changes needed.

4. Health gate

On startup, `openagent-frontend` polls `GET {OPENAGENT_API_URL}/health` every 3 seconds in a blocking while-loop until the response body's `status` field is `"ok"`. The chat input is not rendered until this gate clears. `openagent-api` translates the upstream's `degraded` (provider worker cold-starting) into `loading`, so this layer only needs to recognise three values: `ok`, `loading`, `unreachable`.

States the gate handles:

- `{"status": "ok"}` → gate clears, UI unlocks
- `{"status": "loading"}` → live "⌚ The upstream model is starting up" banner, cold-start narrative
- `{"status": "unreachable"}` → live "🚫 openagent-api is up but cannot reach the upstream model" banner
- Connection error to openagent-api → "🚫 Cannot reach openagent-api" banner
- Any other response → "⚠ Unknown /health status" banner

This is a UI-lock concern, not a health-checking concern. The actual health logic (probing `openagent-infra`, mapping its status, deciding what to report) lives upstream in `openagent-api`. This layer just reads the result and decides whether to let the user type.

5. Error display (presentation layer)

`openagent-frontend` owns the user-facing error presentation for any failure on the wire. Emoji prefixes give a glanceable signal:

Prefix	Class	Source
🚫	Connection / network	TCP connect failed, mid-stream disconnect, HTTP 502
⌚	Timeout / loading	Connect timeout, HTTP 503, HTTP 504
🔒	Auth	HTTP 401
⚠	Request validation	HTTP 400, HTTP 422
❌	Unexpected	Anything not matched above

The frontend does not classify upstream errors — `openagent-api` normalises everything to a consistent set of HTTP codes and in-band SSE error events. The frontend just maps each code to a prefix and a user-facing message.

What This Service Does NOT Own

Explicit non-responsibilities, with the current owner of each.

Owned by `openagent-api`

- **The persona / system prompt** → `openagent-api` loads `bio.txt` once at startup and prepends it as the first system message on every `/chat` call. `bio.txt` is not in this repo.
- **OpenAI messages list construction** → `openagent-api`. The frontend sends only user/assistant turns; `openagent-api` prepends the system message before forwarding to `openagent-infra`.
- **Auth boundary to the model layer** → `openagent-api`. The frontend holds `OPENAGENT_API_KEY` only — the secret for the frontend↔`openagent-api` boundary. `INFRA_API_KEY` (api↔infra) and `PROVIDER_API_KEY` (infra↔provider) live in their respective services and never reach this repo.

Owned by `sse_decoder.py` (a local module)

- **Byte-level SSE parsing** → `sse_decoder.py`. The `data:` prefix stripping, `[DONE]` sentinel detection, `[ERROR ...]` sentinel detection, JSON decoding, and routing-by-delta-key all live there, separate from the rendering logic in `app.py`.

Owned elsewhere

- **Model serving / inference** → the BYOC compute provider (proxied by `openagent-infra`)
- **Reasoning effort default** → `openagent-infra` (its `REASONING_EFFORT` env var)
- **Reasoning: `<level>` injection into the system message** → `openagent-infra`

Not implemented

- **Persistent conversation history** → not implemented. State is per browser tab only.
- **Cross-session / cross-device state** → not implemented.
- **User identity / authentication** → not implemented. `OPENAGENT_API_KEY` is a single shared secret.
- **Rate limiting** → not implemented (belongs at a reverse proxy if ever needed).
- **Multi-tenancy** → not supported.
- **CORS policies / external API clients** → not supported (this is a Streamlit UI, not an API).

API Reference

`openagent-frontend` does **not** expose an HTTP API to other services. It is a Streamlit web app, reached via browser. There is no `/api`, no REST surface, no callable endpoints.

The only externally-observable surface is the Streamlit app served at host port 8000. Services that need to interact with the OpenAgent system integrate with `openagent-api` (the gateway), not with `openagent-frontend`.

Outbound HTTP Contracts

`openagent-frontend` is a client of the following endpoints. These contracts are consumed, not provided. Full specs live in `openagent-api`'s datasheet.

POST `{OPENAGENT_API_URL}/chat` — consumed

Request:

```
POST /chat
Content-Type: application/json
X-API-Key: <OPENAGENT_API_KEY>

{
  "messages": [
    {"role": "user",      "content": "..."},
    {"role": "assistant", "content": "..."},
    {"role": "user",      "content": "..."}
  ],
  "reasoning_effort": "medium"
}
```

The `messages` array contains user/assistant turns ONLY. No system message — `openagent-api` prepends the persona server-side. If the frontend accidentally sends a system message, `openagent-api` drops it with a warning log; the request still succeeds.

The `reasoning_effort` field is optional. The frontend sends it when the user has selected a non-default value via the toggle (Quick / Standard / Deep map to `low` / `medium` / `high`); when the user keeps the toggle on "Default", the field is omitted entirely so `openagent-api` / `openagent-infra` apply their server-side default. Pydantic validation upstream rejects any other value with HTTP 422.

Response: `text/event-stream`

Each event is a JSON-encoded OpenAI ChatCompletion chunk — NOT plain text tokens. Chain-of-thought tokens stream first inside `choices[0].delta.reasoning`, then visible answer tokens inside `choices[0].delta.content`, then a final empty-delta chunk with `finish_reason: "stop"`, then the `[DONE]` sentinel.

```
data: {"id":"chatcmpl-...","choices":[{"delta":
{"role":"assistant","content":""},"finish_reason":null}]}

data: {"id":"chatcmpl-...","choices":[{"delta":
{"reasoning":"User"},"finish_reason":null}]}

data: {"id":"chatcmpl-...","choices":[{"delta":
{},"finish_reason":"stop"}]}
```

```

data: {"id":"chatcmpl-...", "choices":[{"delta":{"reasoning":"
asks"},"finish_reason":null}]}

... (more reasoning tokens – chain-of-thought)

data: {"id":"chatcmpl-...", "choices":[{"delta":
{"content":"Hello"},"finish_reason":null}]}

... (more content tokens – visible answer)

data: {"id":"chatcmpl-...", "choices":[{"delta":
{},"finish_reason":"stop"}]}

data: [DONE]

```

`openagent-frontend` does NOT decode this format inline. It hands the raw line iterator to `sse_decoder.decode_sse_stream()`, which yields typed `SSEEvent` objects (`kind="reasoning"`, `kind="content"`, `kind="finish"`, `kind="error"`, `kind="done"`). The chat-rendering loop in `app.py` routes each event by kind to the appropriate UI surface.

Mid-stream errors are surfaced by `openagent-api` as in-band SSE events: `data: [ERROR upstream_status=503]\n\n` followed by `data: [DONE]\n\n`. `sse_decoder.py` recognises these and yields an `SSEEvent(kind="error", error=...)`, which displays the error banner and stops consumption.

Timeout handling: connect timeout 10 seconds, read timeout `None` (unbounded — generation can take several minutes on a cold start or `high` reasoning effort).

`GET {OPENAGENT_API_URL}/health` — consumed

Request:

```

GET /health
X-API-Key: <OPENAGENT_API_KEY>

```

The endpoint is authenticated — `openagent-api` treats `/health` as operational state worth protecting. Same `X-API-Key` as `/chat`.

Response: Always HTTP 200. Body:

```

{
  "status": "ok" | "loading" | "unreachable",
  "openagent_api": {"version": "...", "identity_loaded": true},
  "openagent_infra": {"url": "...", "status": "...", "raw": {}}
}

```

The frontend's gate-open loop reads the top-level `status` field only. The nested objects are diagnostic detail for operators tailing logs.

Top-level status	Frontend behavior
<code>ok</code>	Gate clears; chat input renders.
<code>loading</code>	Banner: "⌚ The upstream model is starting up" with attempt counter.
<code>unreachable</code>	Banner: "🚫 openagent-api is up but cannot reach the upstream model".
anything else	Banner: "⚠️ Unknown /health status".

Timeout handling: 5-second connect/read timeout, polled every 3 seconds during the gate-open loop.

State Model

Per-browser-session state (in `st.session_state`)

Key	Type	Lifetime	Purpose
<code>session_id</code>	<code>str</code> (8-char UUID fragment)	Browser tab	Log correlation only. Makes frontend logs easy to follow per-tab. Not sent to <code>openagent-api</code> (the gateway is stateless per-request).
<code>messages</code>	<code>list[dict]</code>	Browser tab	Full conversation display + payload source. Shape: <code>{"role": str, "content": str}</code> .
<code>model_ready</code>	<code>bool</code>	Browser tab	Health gate flag. Unlocks chat input when <code>True</code> .
<code>initialised</code>	<code>bool</code>	Browser tab	One-shot startup logging flag.
<code>reasoning_effort</code>	<code>str</code>	Browser tab	Currently-selected toggle label (" <code>Default</code> " / " <code>Quick</code> " / " <code>Standard</code> " / " <code>Deep</code> "). Resolved at submit time to the wire value or <code>None</code> to omit the field.

Cached state

None. The persona is owned by `openagent-api`, so there is nothing to cache here.

Persistent state

None. Closing the browser tab or restarting the container loses all state.

Configuration

All runtime configuration is loaded from `.env` at the repository root via `python-dotenv` and `docker-compose`'s `env_file:` directive.

Variable	Required	Default	Description
<code>OPENAGENT_API_URL</code>	Yes	<code>http://localhost:8001</code>	Base URL of <code>openagent-api</code> . No trailing slash.
<code>OPENAGENT_API_KEY</code>	Yes	—	Shared secret for <code>X-API-Key</code> header on <code>/chat</code> and <code>/health</code> . Must match <code>OPENAGENT_API_KEY</code> in <code>openagent-api</code> 's <code>.env</code> byte-for-byte.

`OPENAGENT_API_URL` values by deployment topology:

Scenario	Value
Everything on host, no Docker	<code>http://localhost:8001</code>
Frontend in Docker, <code>openagent-api</code> on host	<code>http://host.docker.internal:8001</code>
Both in Docker, shared external network	<code>http://openagent-api:8001</code>
External deployment	<code>https://api.your-domain.com</code>

The default deployment uses `host.docker.internal:8001` because `openagent-frontend` and `openagent-api` live in separate Docker Compose stacks. The compose file declares `extra_hosts: host.docker.internal:host-gateway` to make this work on Linux as well as Docker Desktop.

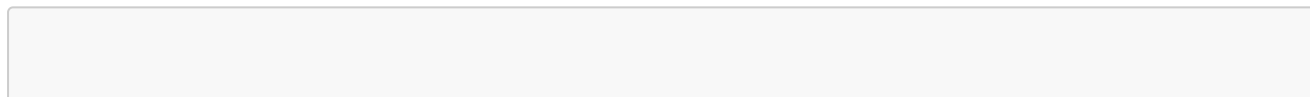
`OPENAGENT_API_KEY` is the FRONTEND ↔ OPENAGENT-API boundary key only. It is **not** the same as `INFRA_API_KEY` (which lives in `openagent-api/.env` and authenticates to `openagent-infra`) or `PROVIDER_API_KEY` (which lives in `openagent-infra/.env` and authenticates to the compute provider). Three independent secrets at three independent boundaries. See `openagent-api`'s security model for the full compartmentalization rationale.

Container / Deployment

Image

- **Base:** `python:3.11-slim`
- **Tag:** `openagent-frontend:1.0.0`
- **Container name:** `openagent-frontend`
- **Size (approximate):** ~450 MB (pure-Python, no CUDA, no BLAS)

Build



```
# From repo root
docker-compose up -d --build
```

The `--build` flag is **required** when `src/frontend/app.py`, `src/frontend/sse_decoder.py`, or the Dockerfile change. Without it Docker reuses the cached image.

Port mapping

- **Host port 8000 → Container port 8501** (Streamlit default internal port)
- Users open `http://localhost:8000`
- Streamlit internally binds to `0.0.0.0:8501`

Volumes

None. The container is stateless (state lives in the browser). No data persists across container restarts.

Restart policy

`unless-stopped` — auto-restarts on crash/OOM, but respects explicit `docker-compose stop`.

File Structure

```
openagent-frontend/
├── docker/
│   └── frontend/
│       └── Dockerfile                # Python 3.11 slim + Streamlit
├── src/
│   └── frontend/
│       ├── app.py                  # The Streamlit UI
│       └── sse_decoder.py          # SSE / ChatCompletion chunk decoder
├── docs/
│   └── DATASHEET.md                # This document
├── docker-compose.yml              # Single-service compose
├── requirements.txt                # streamlit, requests, python-dotenv
├── .env                            # secrets – never committed
├── .env.example                    # template for .env
├── .dockerignore
├── .gitignore
└── README.md
```

Dependency footprint is deliberately tiny: **streamlit**, **requests**, **python-dotenv**, and nothing else.

Integration Notes for Other Services

For openagent-api (the only upstream)

`openagent-api` is the only service `openagent-frontend` talks to. Touchpoints:

- **POST /chat** — frontend sends user/assistant turns plus optional `reasoning_effort`. `openagent-api` prepends the system message and forwards upstream.
- **GET /health** — frontend polls during cold start; `openagent-api` proxies `openagent-infra`'s health and translates `degraded` → `loading`.
- **Auth: X-API-Key: OPENAGENT_API_KEY** on every request to both endpoints.

The contract is documented in `openagent-api`'s datasheet. `openagent-frontend` does not need to know what `openagent-api` does internally; it just consumes the documented endpoints. If `openagent-api`'s request shape changes, this datasheet should be updated and the frontend code with it.

For `openagent-logger` (indirect)

`openagent-frontend` does not talk to `openagent-logger` directly. `openagent-api` emits per-request events on every chat turn, which is the right granularity. The frontend's logs are local and stay local — stdout in the format `%(asctime)s | %(levelname)-8s | %(message)s` (matching `openagent-api` and `openagent-infra` so lines align when tailing all three). The named logger `openagent.frontend` has a child `openagent.frontend.sse_decoder`.

Design Decisions

Why Streamlit?

Streamlit collapses "build a chat UI, style it, add streaming, manage session state, serve it over HTTP" into a single Python file with no JavaScript. The HTTP/SSE contract with `openagent-api` means the frontend can be swapped wholesale later (mobile app, CLI, alternate web framework) without touching the backend.

Why doesn't the frontend own the system prompt?

The persona belongs at the gateway — that's a backend concern, not a UI concern. Keeping it upstream means any client (a mobile app, a CLI) gets the same identity without re-implementing it; a tampered or out-of-date frontend cannot override the persona; and the frontend stops carrying configuration that has no business at the UI layer.

Why is the SSE decoder a separate module?

`sse_decoder.py` is split out of `app.py` for three reasons. First, it isolates JSON-decoding-and-event-routing from Streamlit-and-rendering, so each file is about one thing. Second, it's testable in isolation — `parse_chunk()` takes a string and returns an event, no Streamlit state required. Third, when the upstream chunk format changes, the change is contained to one file. The boundary is clean: transport in (raw line iterator), structured events out (`SSEEvent` dataclasses). `app.py` doesn't import json; `sse_decoder.py` doesn't import streamlit.

Why is conversation history in the frontend?

The full message list is sent on every `/chat` call (`openagent-api` is stateless across requests), so the frontend has to hold the history to send it. There is no persistence layer; that's an accepted limitation for a reference implementation.

Why the collapsible "Show thinking" expander?

Transparency of the reasoning chain is useful while developing and debugging. For a public-facing deployment the expander could be hidden by default — a one-line UI change in `app.py`, no backend modification. The reasoning-format display policy lives at this layer because UX decisions belong with the UI.

Why a blocking health-polling loop?

Streamlit is single-threaded and lacks native auto-refresh. A blocking `while` loop with `st.empty()` status updates is the simplest correct implementation — block until ready, show progress, prevent users from sending requests that will just 503.

Why does the frontend trust openagent-api's error normalisation?

Because `openagent-api` owns the upstream relationship. Re-classifying errors at the frontend would mean duplicating logic that already exists upstream and could disagree with it. `openagent-api` maps upstream conditions onto a small set of HTTP status codes; the frontend just maps each code to an emoji prefix and a message.

Why pure pass-through on `reasoning_effort`?

One source of truth. `openagent-infra`'s `REASONING_EFFORT` env var holds the default; `openagent-api` passes through; the frontend either sets a value or omits the field. Adding a frontend-side default would create two places to check when debugging "why is it always running at medium."

Why the emoji-prefix error scheme?

Consistency. Once you've learned the error semantics in one part of the stack, you can read any log or banner without a legend. The prefixes are also greppable — the same five prefixes mean the same five things top to bottom.

Why port 8501 internal and 8000 external?

8501 is Streamlit's default — keeping it as the container's internal port means zero Streamlit config overrides. 8000 is the user-facing port in the port convention (`openagent-frontend:8000` → `openagent-api:8001` → `openagent-infra:8002` → `provider`). The mapping happens in `docker-compose.yml` where it belongs.

Known Limitations

Context window truncation is not implemented

The frontend forwards whatever messages list it has accumulated; `openagent-api` forwards it to `openagent-infra` without truncation. Long enough conversations will eventually 400 from upstream once the model's context window is exceeded. There is no client-side summarisation or trimming.

Single-user, single-browser session

`st.session_state` is per browser tab. Closing the tab loses the history. No cross-device sync, no multi-user support.

No rate limiting or abuse protection

The frontend trusts its users. If deployed to a wider audience, rate limiting would belong at `openagent-api` or at a reverse proxy in front of it — not at the UI layer.

Cold-start UX is a blocking wait

During an initial serverless worker spin-up, users see a live status banner but cannot do anything else. `openagent-api` correctly reports `loading` during this time, but nothing either layer does can make the worker spin up faster.

Reasoning-format display couples to the upstream chunk format

The reasoning-expander vs answer-bubble split assumes the upstream emits OpenAI ChatCompletion chunks with `delta.reasoning` and `delta.content` channels. If the upstream chunk format ever changes, `sse_decoder.py` is the file that needs updating — not `app.py`. The display-policy code in `app.py` just routes typed events; it doesn't know about JSON.

`OPENAGENT_API_KEY` is a static shared secret

One key, one client. There is no concept of "user A vs user B" at this layer — anyone with the key can use the system fully. Public exposure must wait for an auth layer in front.

openagent-frontend — part of the OpenAgent system